

形式语言与自动机

Formal Languages and Automata Theory

正则语言的性质

计算机科学与技术学院
哈尔滨工业大学（深圳）

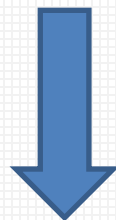


郑宜峰

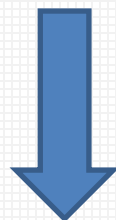
- 正则语言的泵引理
- 正则语言的封闭性
- 正则语言的判定性质
- 自动机的最小化

- **正则语言的泵引理**
- 正则语言的封闭性
- 正则语言的判定性质
- 自动机的最小化

- 思考：如何证明一个语言L不是正则语言？
 - 证明没有任何一个DFA 或 NFA 或 RE能够接受L

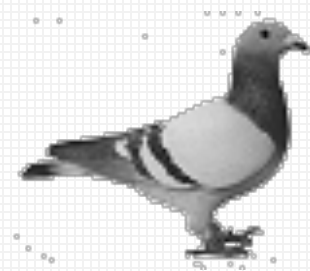
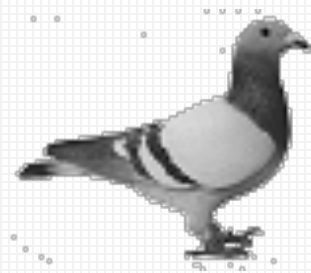
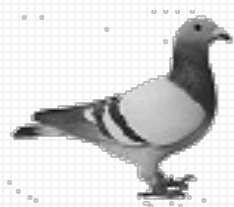
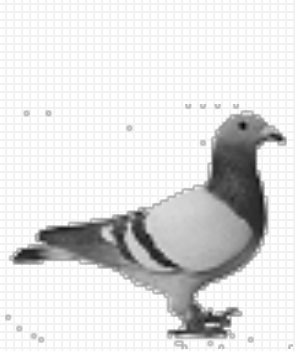


- 难点：实现困难，需要证明无数DFA/NFA/RE

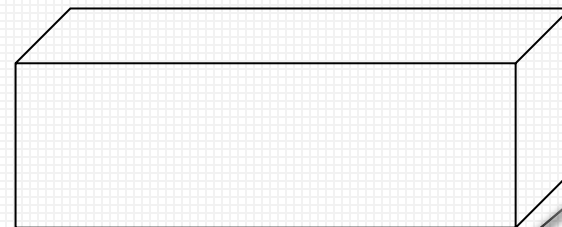
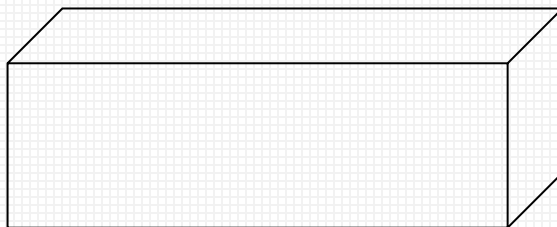
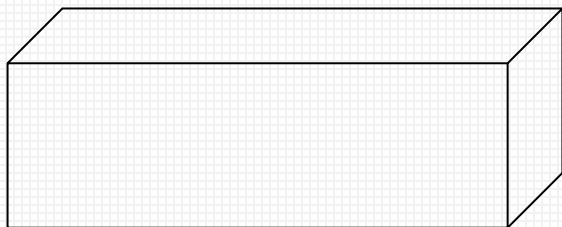


- 解决方法：使用泵引理（Pumping Lemma）

- 在学习泵引理之前，我们首先学习鸽巢原理。

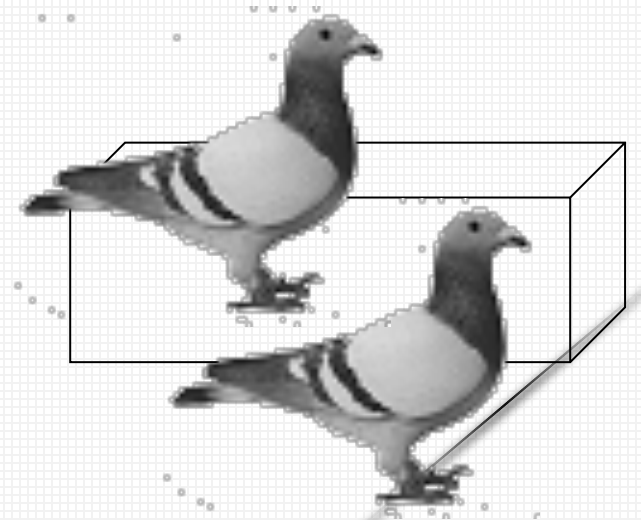
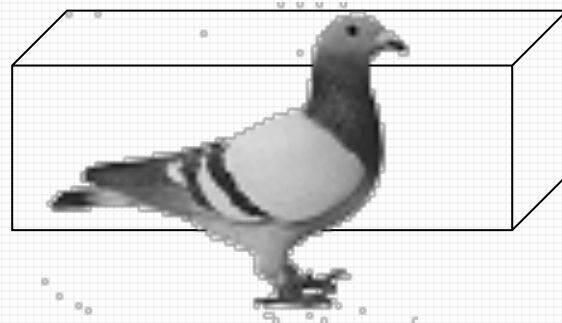
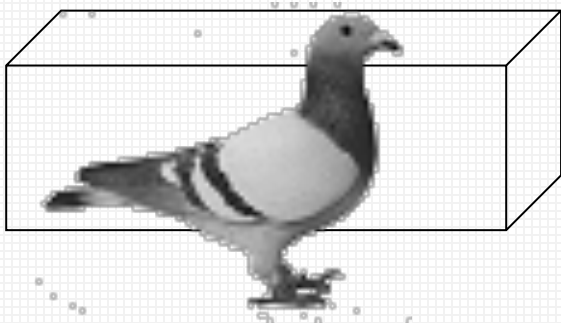


4只鸽子



3个鸽巢

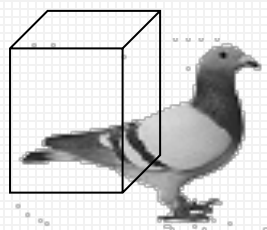
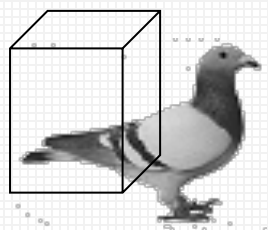
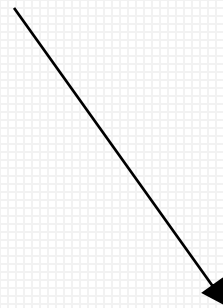
- 如果要把每只鸽子都放到鸽巢中，那么必定有一个鸽巢中存在至少两只鸽子。



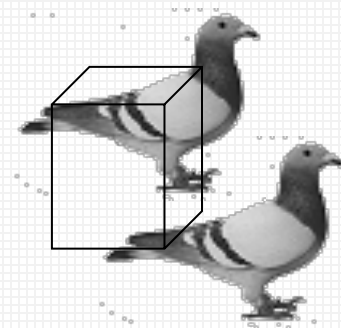
n 只鸽子
 m 个鸽巢

$$n > m$$

必定有一个鸽巢中
存在至少两只鸽子



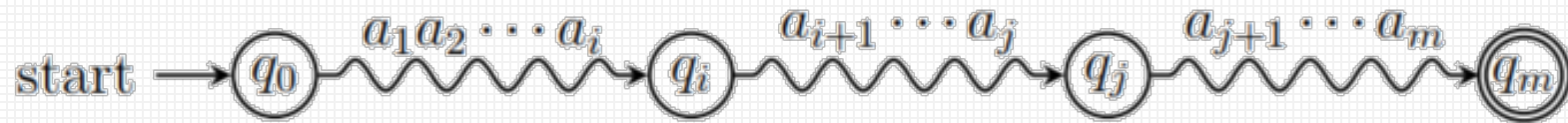
.....



有限状态、无限字符串的情况下，
会是什么情况呢？

- 泵引理 (pumping lemma): 如果语言 L 是正则的, 那么存在正整数 N , 对 $w \in L$, 只要 $|w| \geq N$, 就可以将 w 分为三部分 $w = xyz$ 满足:
 - $y \neq \varepsilon$ (或 $|y| > 0$);
 - $|xy| \leq N$;
 - $\forall k \geq 0, xy^kz \in L$;
 - 中间的串不论循环几次, 所得的串仍属于该正则语言。

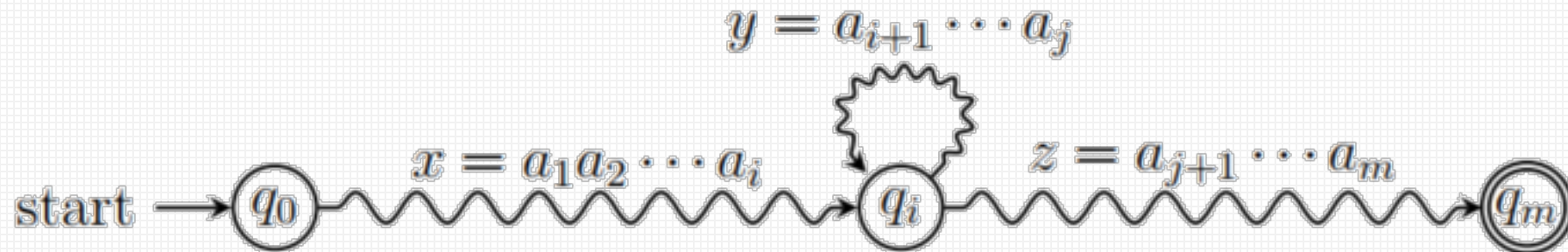
- 如果 L 正则, 那么存在有 N 个状态的DFA A 使 $L(A) = L$;
- 取 $w = a_1 a_2 \cdots a_m \in L$ ($m \geq N$), 定义 $q_i = \hat{\delta}(q_0, a_1 \cdots a_i)$, 则 $q_m = \hat{\delta}(q_0, w) = \hat{\delta}(q_0, a_1 \cdots a_m)$



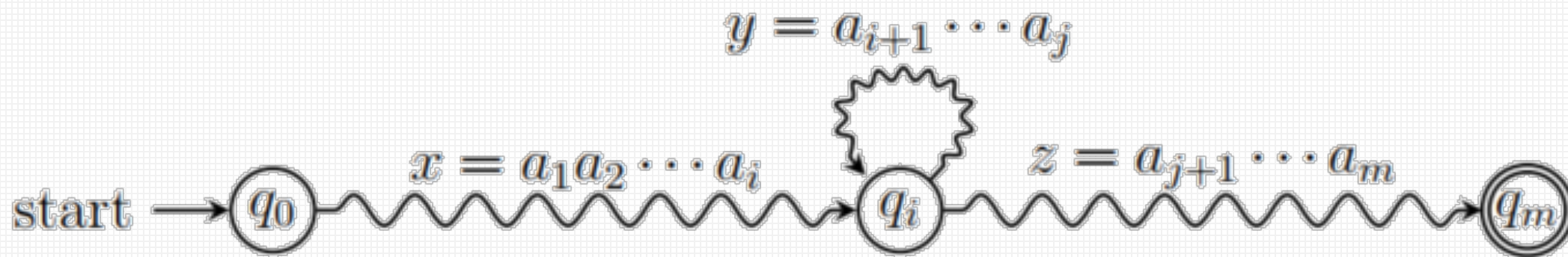
- 当输入 w 的前 N 个字符时, 经过的状态分别有 $q_0, q_1, \dots, q_N$ ($N+1$ 个状态)
- 因为DFA只有 N 个状态, 由鸽巢原理 (pigeonhole principle), 所到达的状态必有两个相同
 - 鸽巢原理: $N + 1$ 只鸽子飞入 N 个巢, 则至少有一个巢有不少于两只鸽子。

- 不妨设 $q_i = q_j$ ($0 \leq i < j \leq N$)
 - 因为 q_i 的定义是每次读入一个字符到达的状态，所以 i 和 j 至少相差一个字符， i 是不等于 j 的。
 - 不妨设 $i < j$ 。因为只看 w 的前 N 个字符，所以 $j \leq N$ 。

- 不妨设 $q_i = q_j$ ($0 \leq i < j \leq N$)
- 根据 q_i 和 q_j , 可以把 w 这个路径如图分为三段, 分别是 $x = a_1 a_2 \cdots a_i$, $y = a_{i+1} \cdots a_j$, $z = a_{j+1} \cdots a_m$
 - 一段从 q_0 开始, 经过 x , 到达 q_i
 - 然后经过 y , 到达 q_j , 但 $q_i = q_j$, 所以实际到达 q_i
 - 最后经过 z , 还是到达 q_m



- $\forall k \geq 0, xy^kz \in L$;
 - 在 y 处做循环
- 因为 $i < j$, x 和 y 之间至少有一个字符, 故 $y \neq \varepsilon$ (即 $|y| > 0$) ;
- 因为 $j \leq N$ 所以 $|xy| \leq N$ (即重复状态的出现必然在输入前 N 个符号的过程中出现)



- **泵引理的解释**：任何从开始状态到接受状态的路径，如果长度超过 N ，经过大于 N 个状态，必定有一个重复状态，因此会形成一个循环（loop）；那么，这个循环可以被重复多次后，沿原路径还会到达接收状态。
- **泵引理可以用来确定特定语言不是正则语言**，泵引理成立是正则语言判定的**必要条件**，但不是充分条件
 - 即如果一个语言不满足泵引理，则其不是正则语言

- 使用泵引理证明某个语言 L 不是正则的
 - 先假设语言 L 是正则的，则一定存在正整数 N ，对 $w \in L(|w| \geq N)$ 满足泵引理。
 - 选取 L 中某个满足长度要求的符号串 w ，分解 $w = xyz$
 - 证明存在某个 $k \geq 0$ ，使得 xy^kz 不属于 L
 - 而由泵引理，如果 L 是正则语言，则 xy^kz 应该属于 L 。
 - 存在矛盾，因此得出结论，语言 L 不是正则的

- 例1: 证明 $L_{01} = \{0^n 1^n | n \geq 0\}$ 不是正则语言。
- 证明: 假设 L_{01} 是正则的,
 - 那么一定存在正整数 N , 对 $w \in L_{01} (|w| \geq N)$ 满足泵引理。

对 $w \in L$, 只要 $|w| \geq N$, 就可以将 w 分为三部分 $w = xyz$ 满足:

$y \neq \varepsilon$ (或 $|y| > 0$);

$|xy| \leq N$;

$\forall k \geq 0, xy^kz \in L$;

- 例1: 证明 $L_{01} = \{0^n 1^n \mid n \geq 0\}$ 不是正则语言。
- 证明: 假设 L_{01} 是正则的,
 - 那么一定存在正整数 N , 对 $w \in L_{01} (|w| \geq N)$ 满足泵引理。
 - 从 L_{01} 中取 $w = 0^N 1^N$, 显然 $w \in L_{01}$, 且 $|w| = 2N > N$ 。

- 例1: 证明 $L_{01} = \{0^n 1^n \mid n \geq 0\}$ 不是正则语言。
- 证明: 假设 L_{01} 是正则的,
 - 那么一定存在正整数 N , 对 $w \in L_{01} (|w| \geq N)$ 满足泵引理。
 - 从 L_{01} 中取 $w = 0^N 1^N$, 显然 $w \in L_{01}$, 且 $|w| = 2N > N$ 。
 - 那么, w 可被分为 $w = xyz$, 且 $|xy| \leq N$ 和 $y \neq \varepsilon$ 。

- 例1: 证明 $L_{01} = \{0^n 1^n \mid n \geq 0\}$ 不是正则语言。
 - 证明: 假设 L_{01} 是正则的,
 - 那么一定存在正整数 N , 对 $w \in L_{01} (|w| \geq N)$ 满足泵引理。
 - 从 L_{01} 中取 $w = 0^N 1^N$, 显然 $w \in L_{01}$, 且 $|w| = 2N > N$ 。
 - 那么, w 可被分为 $w = xyz$, 且 $|xy| \leq N$ 和 $y \neq \varepsilon$ 。
 - 因为 $|xy| \leq N$, 令 $x = 0^m, y = 0^n, z = 0^{N-m-n} 1^N$, 且 $n > 0$
 - 那么 $xy^0z = 0^{N-n} 1^N \notin L_{01}$, 而由泵引理, $xy^0z \in L_{01}$, 矛盾。
- 所以假设不成立, L_{01} 不是正则的。

- 例2: 证明 $L = \{0^n 1^m 0^n \mid m, n \geq 0\}$ 不是正则的。

证明: 假设 L 是正则的

- 那么一定存在正整数 N , 对 $w \in L$ ($|w| \geq N$) 满足泵引理。
- 从 L 中取 $w = 0^N 1 0^N$, 显然 $w \in L$, 且 $|w| = (2N + 1) > N$ 。
- 那么, w 可被分为 $w = xyz$, 且 $|xy| \leq N$ 和 $y \neq \varepsilon$ 。
- 因此, 令 $x = 0^m, y = 0^n, z = 0^{N-m-n} 1 0^N$, 且 $n > 0$
- 那么, $xy^2z = 0^{N+n} 1 0^N$ 。由于 $n > 0, N + n > N$, 所以 xy^2z 不属于 L , 而由泵引理, 应该要有 $xy^2z \in L$, 矛盾。

所以假设不成立, L 不是正则的。

- 例3: 证明 $L = \{0^i 1^j \mid i > j\}$ 不是正则的。
- 证明: 假设 L 是正则的
 - 那么一定存在正整数 N , 对 $w \in L$ ($|w| \geq N$) 满足泵引理。

- 例3: 证明 $L = \{0^i 1^j \mid i > j\}$ 不是正则的。
- 证明: 假设 L 是正则的
 - 那么一定存在正整数 N , 对 $w \in L$ ($|w| \geq N$) 满足泵引理。
 - 从 L 中取 $w = 0^{N+1} 1^N$, 显然 $w \in L$, 且 $|w| = 2N + 1 > N$ 。

- 例3: 证明 $L = \{0^i 1^j \mid i > j\}$ 不是正则的。
 - 证明: 假设 L 是正则的
 - 那么一定存在正整数 N , 对 $w \in L$ ($|w| \geq N$) 满足泵引理。
 - 从 L 中取 $w = 0^{N+1} 1^N$, 显然 $w \in L$, 且 $|w| = 2N + 1 > N$ 。
 - 那么, w 可被分为 $w = xyz$, 且 $|xy| \leq N$ 和 $y \neq \varepsilon$ 。
 - 因此, y 只能是 0^m 且 $m > 0$ (即 $m \geq 1$)。
 - 那么, $xy^0z = 0^{N+1-m} 1^N \notin L_{01}$, 因为 $N + 1 - m \leq N$, 而由泵引理, $xy^0z \in L$, 矛盾。
- 所以假设不成立, L 不是正则的。

- 正则语言的泵引理
- **正则语言的封闭性**
- 正则语言的判定性质
- 自动机的最小化

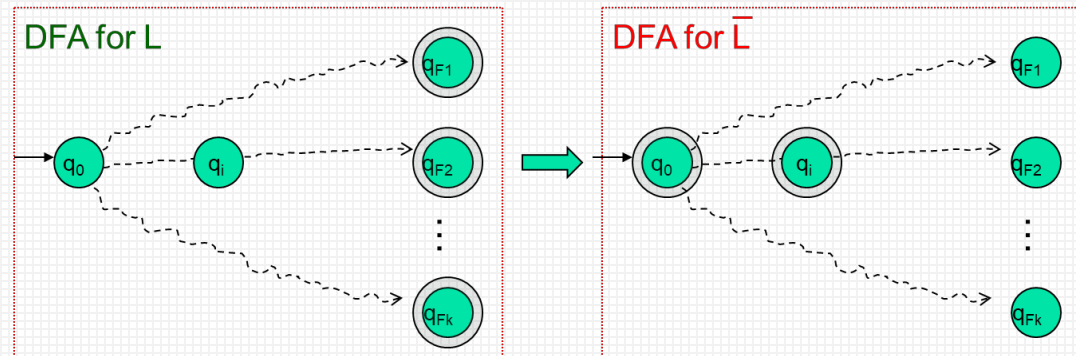
- 正则语言经某些**运算**后得到的新语言仍保持正则，称正则语言在这些运算下**封闭**。
- 正则语言 L 和 M ，在这些运算下封闭：
 - 并: $L \cup M$ ，连接: $L \cdot M$ ，闭包: L^*
 - 补: $\bar{L}$
 - 交: $L \cap M$
 - 差: $L - M$
 - 反转: $L^R = \{w^R \mid w \in L\}$
 - 同态: $h(L) = \{h(w) \mid w \in L\}$, 同态: $h: \Sigma \rightarrow \Gamma^*$
 - 逆同态: $h^{-1}(L) = \{w \in \Sigma^* \mid h(w) \in L \subset \Gamma^*, \text{同态: } h: \Sigma \rightarrow \Gamma^*\}$

- 正则语言经某些**运算**后得到的新语言仍保持正则，称正则语言在这些运算下**封闭**。
- 正则语言 L 和 M ，在这些运算下封闭：
 - 并: $L \cup M$ ，连接: $L \cdot M$ ，闭包: L^*
 - 补: $\bar{L}$
 - 交: $L \cap M$
 - 差: $L - M$
 - 反转: $L^R = \{w^R \mid w \in L\}$
 - 同态: $h(L) = \{h(w) \mid w \in L\}$, 同态: $h: \Sigma \rightarrow \Gamma^*$
 - 逆同态: $h^{-1}(L) = \{w \in \Sigma^* \mid h(w) \in L \subset \Gamma^*, \text{同态: } h: \Sigma \rightarrow \Gamma^*\}$

- 定理：正则语言在并，连接和闭包运算下保持封闭。
- 证明：由正则表达式的定义得证。
 - 比如说， L 和 M 如果是正则语言，那么一定存在正则表达式 r 和 m 去表示它们，即 $L = L(r), M = L(m)$
 - 则有， $L \cup M = L(r + m), L \cdot M = L(rm), L^* = L(r^*)$
 - 即运算结果存在对应的正则表达式

- 正则语言经某些**运算**后得到的新语言仍保持正则，称正则语言在这些运算下**封闭**。
- 正则语言 L 和 M ，在这些运算下封闭：
 - 并: $L \cup M$ ，连接: $L \cdot M$ ，闭包: L^*
 - 补: $\bar{L}$
 - 交: $L \cap M$
 - 差: $L - M$
 - 反转: $L^R = \{w^R \mid w \in L\}$
 - 同态: $h(L) = \{h(w) \mid w \in L\}$, 同态: $h: \Sigma \rightarrow \Gamma^*$
 - 逆同态: $h^{-1}(L) = \{w \in \Sigma^* \mid h(w) \in L \subset \Gamma^*, \text{同态: } h: \Sigma \rightarrow \Gamma^*\}$

- **定理:** 如果 L 是 Σ 上的正则语言, 那么 $\bar{L} = \Sigma^* - L$ 也是正则的。(求补就是把 L 对应的字符串从 Σ^* 中去掉。)
- **证明:** 设接受语言 L 的 DFA $A = (Q, \Sigma, \delta, q_0, F)$, 即 $L(A) = L$.
构造 DFA $B = (Q, \Sigma, \delta, q_0, Q - F)$, 即把 A 的所有接受状态改成非接受状态, 非接受状态改成接受状态。



- 则有 $\bar{L} = L(B)$, 因为 $\forall w \in \Sigma^*, w \in \bar{L} \iff \hat{\delta}(q_0, w) \notin F \iff \hat{\delta}(q_0, w) \in Q - F \iff w \in L(B)$ (因为在 B 中, $Q - F$ 为接受状态集)

- 例：证明 $L_{neq} = \{w \mid w \text{ 由数量不相等的0 和1 构成} \}$ 不是正则的。

- **例：证明** $L_{neq} = \{w \mid w \text{由数量不相等的0 和1 构成}\}$ **不是正则的。**
 - 由泵引理很难直接证明 L_{neq} 不是正则的。
 - 比如如果选取一个符号串 w ，将其打断为 $w = xyz$ 形式，然后利用 y 产生新的串。但是， y 本身可能包含相同数量的0和1，比如01串。如果基于 y 去泵出新的串，如果 xyz 本身0和1数量不相等，且 y 包含数量相等的0，1，那么产生的新串中0和1的数量还是不相等，应该泵出来的串还是会属于 L_{neq} 。即无法通过矛盾来完成泵引理的证明。

- **例：证明** $L_{neq} = \{w \mid w \text{由数量不相等的0和1构成}\}$ **不是正则的。**
 - 由泵引理很难直接证明 L_{neq} 不是正则的。
 - 比如如果选取一个符号串 w ，将其打断为 $w = xyz$ 形式，然后利用 y 产生新的串。但是， y 本身可能包含相同数量的0和1，比如01串。如果基于 y 去泵出新的串，如果 xyz 本身0和1数量不相等，且 y 包含数量相等的0，1，那么产生的新串中0和1的数量还是不相等，应该泵出来的串还是会属于 L_{neq} 。即无法通过矛盾来完成泵引理的证明。
 - 考虑 $L_{eq} = \{w \mid w \text{由数量相等的0和1构成}\}$ ，可以看出 $L_{eq} = \overline{L_{neq}}$ ，故 $L_{neq} = \overline{L_{eq}}$ 。
 - 而 L_{eq} 不是正则的，比如前面证明了诸如 $0^n 1^n$ 这样形式的串是不满足泵引理的。
 - 因此，根据补运算的封闭性， L_{neq} 不是正则的。

- 正则语言经某些**运算**后得到的新语言仍保持正则，称正则语言在这些运算下**封闭**。
- 正则语言 L 和 M ，在这些运算下封闭：
 - 并: $L \cup M$ ，连接: $L \cdot M$ ，闭包: L^*
 - 补: $\bar{L}$
 - 交: $L \cap M$
 - 差: $L - M$
 - 反转: $L^R = \{w^R \mid w \in L\}$
 - 同态: $h(L) = \{h(w) \mid w \in L\}$, 同态: $h: \Sigma \rightarrow \Gamma^*$
 - 逆同态: $h^{-1}(L) = \{w \in \Sigma^* \mid h(w) \in L \subset \Gamma^*, \text{同态: } h: \Sigma \rightarrow \Gamma^*\}$

- 定理：如果 L 和 M 是正则语言，那么 $L \cap M$ 也是正则语言。
- 证明1：由 $L \cap M = \overline{\overline{L} \cup \overline{M}}$ 得证。（并运算和补运算均满足封闭性）
- 证明2：通过DFA构造来证明。

- 定理：若DFA A_L , A_M 和 A 的定义如下

$$A_L = (Q_L, \Sigma, \delta_L, q_L, F_L), \quad A_M = (Q_M, \Sigma, \delta_M, q_M, F_M)$$

$$A = (Q_L \times Q_M, \Sigma, \delta, (q_L, q_M), F_L \times F_M)$$

其中, $\delta: (Q_L \times Q_M) \times \Sigma \rightarrow Q_L \times Q_M$, 即:

$$\delta((p, q), a) = (\delta_L(p, a), \delta_M(q, a))$$

则对任意 $w \in \Sigma^*$, $\hat{\delta}((q_L, q_M), w) = (\hat{\delta}_L(q_L, w), \hat{\delta}_M(q_m, w))$

即, $\hat{\delta}((q_L, q_M), w)$ 到达的状态, 和 $\hat{\delta}_L(q_L, w)$ 和 $\hat{\delta}_M(q_m, w)$ 组成的二元组是相等的。

• 证明：通过对 w 的归纳来证明

归纳基础：当 $w = \varepsilon$ 时，

$$\begin{aligned}\hat{\delta}((q_L, q_M), \varepsilon) &= (q_L, q_M) \\ &= (\hat{\delta}_L(q_L, \varepsilon), \hat{\delta}_M(q_M, \varepsilon))\end{aligned}$$

$\hat{\delta}$ 的定义

$\hat{\delta}$ 的定义

假设 $w = x$ 的时候成立， $\hat{\delta}((q_L, q_M), x) = (\hat{\delta}_L(q_L, x), \hat{\delta}_M(q_M, x))$

归纳递推：当 $w = xa$ 时，

$$\begin{aligned}\hat{\delta}((q_L, q_M), w) &= \delta(\hat{\delta}((q_L, q_M), x), a) \\ &= \delta((\hat{\delta}_L(q_L, x), \hat{\delta}_M(q_M, x)), a) \\ &= (\delta_L(\hat{\delta}_L(q_L, x), a), \delta_M(\hat{\delta}_M(q_M, x), a)) \\ &= (\hat{\delta}_L(q_L, xa), \hat{\delta}_M(q_M, xa))\end{aligned}$$

$\hat{\delta}$ 的定义

归纳假设

δ 的构造

$\hat{\delta}$ 的定义

δ 的构造：

$$\begin{aligned}\delta((p, q), a) &= \\ &(\delta_L(p, a), \delta_M(q, a))\end{aligned}$$

要证明

$$\hat{\delta}((q_L, q_M), w) = (\hat{\delta}_L(q_L, w), \hat{\delta}_M(q_M, w))$$

- 定理：如果 L 和 M 是正则语言，那么 $L \cap M$ 也是正则语言。
- 证明2：由上一个定理构造 $L \cap M$ 的DFA A ，则对 $w \in \Sigma^*$

$$\begin{aligned}w \in L \cap M &\iff \hat{\delta}_L(q_L, w) \in F_L \wedge \hat{\delta}_M(q_M, w) \in F_M \\&\iff (\hat{\delta}_L(q_L, w), \hat{\delta}_M(q_M, w)) \in F_L \times F_M \\&\iff \hat{\delta}((q_L, q_M), w) \in F_L \times F_M \\&\iff w \in \mathbf{L}(A).\end{aligned}$$

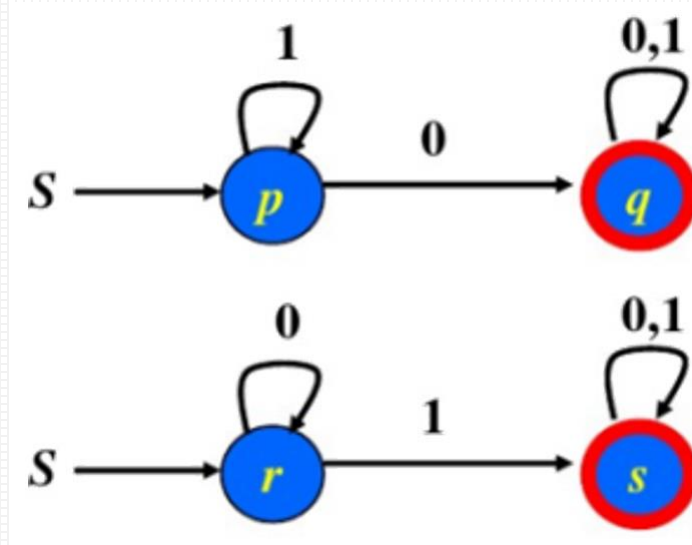
• 示例: $L_1 = \{\text{由}\{0,1\}\text{组成的至少含有一个0的字符串}\}$

$L_2 = \{\text{由}\{0,1\}\text{组成的至少含有一个1的字符串}\}$

求 $L_1 \cap L_2$

$L_1 : 1^*0(0+1)^*$

$L_2 : 0^*1(0+1)^*$



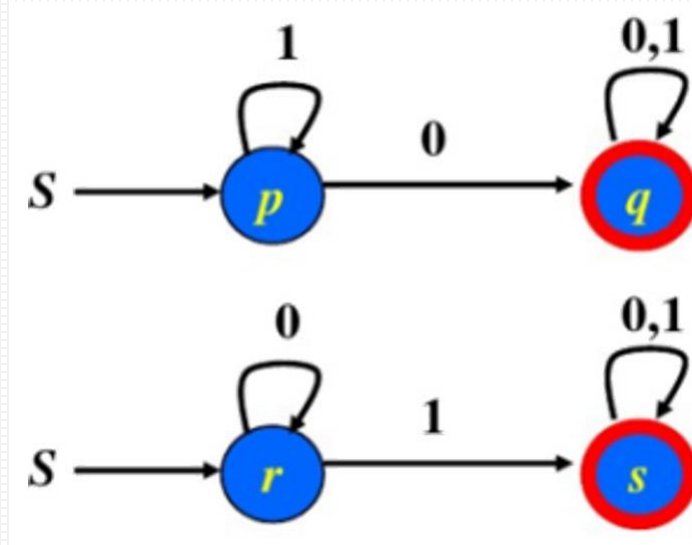
• 示例: $L_1 = \{\text{由}\{0,1\}\text{组成的至少含有一个0的字符串}\}$

$L_2 = \{\text{由}\{0,1\}\text{组成的至少含有一个1的字符串}\}$

求 $L_1 \cap L_2$

$L_1 : 1^*0(0+1)^*$

$L_2 : 0^*1(0+1)^*$



$L_1 \cap L_2$

	0	1		0	1
$\rightarrow pr$	qr	ps	ps	qs	ps
qr	qr	qs	$*qs$	qs	qs

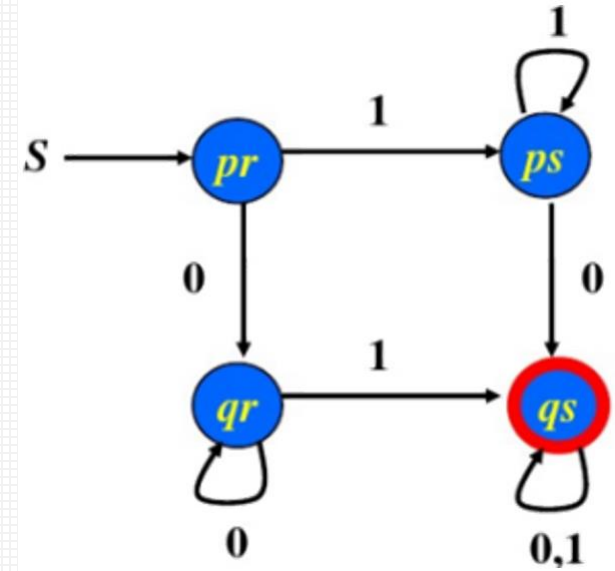
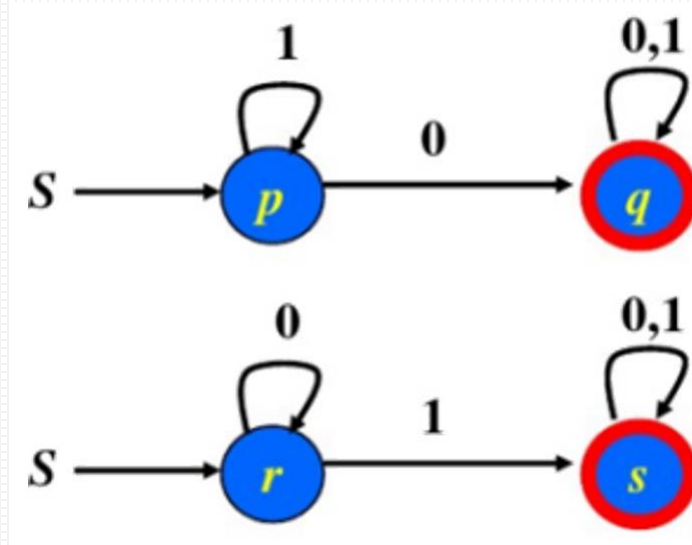
- 示例: $L_1 = \{\text{由}\{0,1\}\text{组成的至少含有一个0的字符串}\}$

$L_2 = \{\text{由}\{0,1\}\text{组成的至少含有一个1的字符串}\}$

求 $L_1 \cap L_2$

$L_1: 1^*0(0+1)^*$

$L_2: 0^*1(0+1)^*$



$L_1 \cap L_2$

	0	1		0	1
$\rightarrow pr$	qr	ps	ps	qs	ps
qr	qr	qs	$*qs$	qs	qs

- 例：如果已知语言 $L_{01} = \{0^n 1^n | n \geq 0\}$ 不是正则的，请用封闭性证明语言 $L_{eq} = \{w | w \text{ 由数量相等的0和1构成}\}$ 也不是正则的。
- 证明：

- 例：如果已知语言 $L_{01} = \{0^n 1^n | n \geq 0\}$ 不是正则的，请用封闭性证明语言 $L_{eq} = \{w | w \text{ 由数量相等的 } 0 \text{ 和 } 1 \text{ 构成}\}$ 也不是正则的。
- 证明：
 - 首先, 因为 0^*1^* 是正则语言（ 0^* 是正则表达式， 1^* 是正则表达式，则根据封闭性， 0^*1^* 也是正则表达式）
 - 而 $L_{01} = L(0^*1^*) \cap L_{eq}$;
 - 如果 L_{eq} 是正则的, L_{01} 必然也是正则的;
 - 因为已知 L_{01} 不是正则的, 所以 L_{eq} 一定不是正则的

- 例: 如果 L_1 和 L_2 都不是正则的, 那么 $L_1 \cap L_2$ 一定不是正则的吗?
- 不一定. 因为, 如果令

$$L_1 = \{0^n 1^n \mid n \geq 0\}$$

$$L_2 = \{a^n b^n \mid n \geq 0\}$$

- 显然两者都不算正则语言, 但

$$L_1 \cap L_2 = \{\varepsilon\}$$

是正则语言。

- 正则语言经某些**运算**后得到的新语言仍保持正则，称正则语言在这些运算下**封闭**。
- 正则语言 L 和 M ，在这些运算下封闭：
 - 并: $L \cup M$ ，连接: $L \cdot M$ ，闭包: L^*
 - 补: $\bar{L}$
 - 交: $L \cap M$
 - 差: $L - M$
 - 反转: $L^R = \{w^R \mid w \in L\}$
 - 同态: $h(L) = \{h(w) \mid w \in L\}$, 同态: $h: \Sigma \rightarrow \Gamma^*$
 - 逆同态: $h^{-1}(L) = \{w \in \Sigma^* \mid h(w) \in L \subset \Gamma^*, \text{同态: } h: \Sigma \rightarrow \Gamma^*\}$

- 定理: 如果 L 和 M 都是正则语言, 那么 $L - M$ 也是正则的.
- 证明: $L - M = L \cap \bar{M}$
 - 正则语言在交运算和补运算下是封闭的。

- 正则语言经某些**运算**后得到的新语言仍保持正则，称正则语言在这些运算下**封闭**。
- 正则语言 L 和 M ，在这些运算下封闭：
 - 并: $L \cup M$ ，连接: $L \cdot M$ ，闭包: L^*
 - 补: $\bar{L}$
 - 交: $L \cap M$
 - 差: $L - M$
 - 反转: $L^R = \{w^R \mid w \in L\}$
 - 同态: $h(L) = \{h(w) \mid w \in L\}$, 同态: $h: \Sigma \rightarrow \Gamma^*$
 - 逆同态: $h^{-1}(L) = \{w \in \Sigma^* \mid h(w) \in L \subset \Gamma^*, \text{同态: } h: \Sigma \rightarrow \Gamma^*\}$

- 定义：字符串 $w = a_1 a_2 \dots a_n$ 的反转，记为 w^R ，定义为 $w^R = a_n a_{n-1} \dots a_1$
- 语言 L 的反转，记为 L^R ，定义为
$$L^R = \{w^R \in \Sigma^* \mid w \in L\}$$

- 定理：如果 L 是正则语言，那么 L^R 也是正则的。
- 证明：假设 L 是由正则表达式 E 描述的语言($L = L(E)$)，对 E 进行归纳，证明存在正则表达式 E^R ，且 $L(E^R) = (L(E))^R$ 。

归纳基础：如果 E 分别是 ε ， $\emptyset$ 或 a ，则 $\varepsilon^R = \varepsilon$ ， $\emptyset^R = \emptyset$ ， $a^R = a$

归纳递推：分三种情况考虑：

- 1. $E = E_1 + E_2$, 则 $E^R = E_1^R + E_2^R$, 对应的语言 $\mathbf{L}(E^R) = \mathbf{L}(E)^R = \mathbf{L}(E_1 + E_2)^R = (\mathbf{L}(E_1) \cup \mathbf{L}(E_2))^R = \mathbf{L}(E_1)^R \cup \mathbf{L}(E_2)^R$

归纳递推：分三种情况考虑：

- 2. $E = E_1E_2$, 则 $E^R = E_2^R E_1^R$, 对应的语言 $L(E^R) = L(E)^R = L(E_1E_2)^R = (L(E_1) \cdot L(E_2))^R = L(E_2)^R L(E_1)^R$

比如, 若 $L(E_1) = \{01, 111\}$, $L(E_2) = \{00, 10\}$, 则 $L(E_1E_2) = \{0100, 0110, 11100, 11110\}$, $L(E_1E_2)^R = \{0010, 0110, 00111, 01111\}$ 。对于 $L(E_2)^R L(E_1)^R$, 有 $\{00, 01\} \{10, 111\} = \{0010, 00111, 0110, 01111\}$ 。可以看出, $L(E_1E_2)^R = L(E_2)^R L(E_1)^R$

归纳递推：分三种情况考虑：

- 3. $E = E_1^*$ ，则 $E^R = (E_1^R)^*$ ，对应的语言 $L(E^R) = L(E)^R = L(E_1^*)^R = L((E_1^R)^*)$
 - $L(E)$ 中的任意一个符号串 w 可以表示为 $w = w_1 w_2 \dots w_n$ ，每个 w_i 属于 $L(E_1)$ 。对 w 求反转，则有 $w^R = w_n^R w_{n-1}^R \dots w_1^R$ ，每个 w_i^R 属于 $L(E_1^R)$ ，因此 w^R 属于 $L((E_1^R)^*)$
 - 反之， $L((E_1^R)^*)$ 中的任意一个符号串 $w = w_1 w_2 \dots w_n$ ，每个 w_i 属于 $L(E_1^R)$ 。对 w 求反转，则有 $w^R = w_n^R w_{n-1}^R \dots w_1^R$ ，每个 w_i^R 属于 $L(E_1)$ ，因此 w^R 属于 $L(E_1^*)$ ，即 $L(E)$
 - 因此，一个符号串 w 属于 $L(E)$ ，当且仅当 w^R 属于 $L((E_1^R)^*)$ ，即 $L(E^R) = L((E_1^R)^*)$

- 例：设语言 L 的正则表达式为 $(0 + 1)0^*$ ，则 L^R 是？

- 例：设语言 L 的正则表达式为 $(0 + 1)0^*$ ，则 L^R 是？
- 解：由(1)若 $E = E_1E_2$ ，则 $E^R = E_2^R E_1^R$ ，对 $(0 + 1)0^*$ 求反转，有 $(0^*)^R(0 + 1)^R$ ；
 - 由(3)若 $E = E_1^*$ ，则 $E^R = (E_1^R)^*$ ，有 $(0^*)^R = (0^R)^*$ ；因为 $0^R = 0$ ，所以 $(0^*)^R = (0^R)^* = 0^*$ 。
 - 由(1)若 $E = E_1 + E_2$ ，则 $E^R = E_1^R + E_2^R$ ，有 $(0 + 1)^R = 0^R + 1^R$ ；因为 $0^R = 0, 1^R = 1$ ，所以 $(0 + 1)^R = 0 + 1$
 - 因此， $L^R = (0^*)^R(0 + 1)^R = 0^*(0 + 1)$

- 正则语言经某些**运算**后得到的新语言仍保持正则，称正则语言在这些运算下**封闭**。
- 正则语言 L 和 M ，在这些运算下封闭：
 - 并: $L \cup M$ ，连接: $L \cdot M$ ，闭包: L^*
 - 补: $\bar{L}$
 - 交: $L \cap M$
 - 差: $L - M$
 - 反转: $L^R = \{w^R \mid w \in L\}$
 - 同态: $h(L) = \{h(w) \mid w \in L, \text{同态: } h: \Sigma \rightarrow \Gamma^*\}$
 - 逆同态: $h^{-1}(L) = \{w \in \Sigma^* \mid h(w) \in L \subset \Gamma^*, \text{同态: } h: \Sigma \rightarrow \Gamma^*\}$

- 定义:若 Σ 和 Γ 是两个字母表, 同态定义为函数 $h : \Sigma \rightarrow \Gamma^*$ (即将 Σ 上的一个符号映射为 Γ 上的符号串)

$$\forall a \in \Sigma, h(a) \in \Gamma^*.$$

- 扩展 h 的定义到字符串,

$$(1) h(\varepsilon) = \varepsilon$$

$$(2) h(xa) = h(x)h(a)$$

- 再扩展 h 到语言, 对 $\forall L \subseteq \Sigma^*$,

$$h(L) = \{h(w) \mid w \in L\}.$$

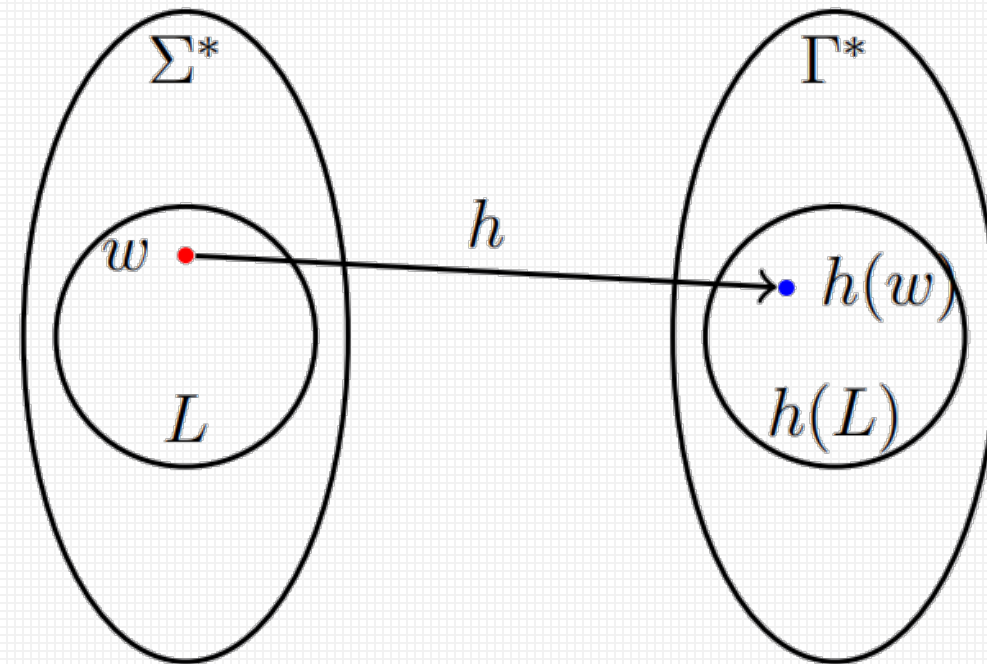
- 例: 若由 $\Sigma = \{0, 1\}$ 到 $\Gamma = \{a, b\}$ 的同态函数 h 定义为 $h(0) = ab, h(1) = \varepsilon$, 则 Σ 上的字符串 0011, 在 h 的作用下

$$\begin{aligned}h(0011) &= h(0)h(0)h(1)h(1) \\&= abab\varepsilon\varepsilon \\&= abab\end{aligned}$$

- 语言 $L = 1^*0 + 0^*1$, 在 h 的作用下, $h(L)$ 为:

$$\begin{aligned}h(1^*0 + 0^*1) &= (h(1))^*h(0) + (h(0))^*h(1) \\&= (\varepsilon)^*(ab) + (ab)^*(\varepsilon) \\&= (ab)^*\end{aligned}$$

- 定理：若 L 是字母表 Σ 上的正则语言， h 是 Σ 上的同态，则 $h(L)$ 也是正则的。



- 若 L 的正则表达式为 E , 即 $L = L(E)$, 按如下规则定义构造表达式 $h(E)$:

$$h(\emptyset) = \emptyset$$

$$h(\mathbf{r} + \mathbf{s}) = h(\mathbf{r}) + h(\mathbf{s})$$

$$h(\epsilon) = \epsilon$$

$$h(\mathbf{rs}) = h(\mathbf{r})h(\mathbf{s})$$

$$\forall a \in \Sigma, h(\mathbf{a}) = h(a)$$

$$h(\mathbf{r}^*) = (h(\mathbf{r}))^*$$

- 往证 $h(L(E)) = L(h(E))$, 而 $h(E)$ 显然也是正则表达式, 因此 $h(L)$ 正则。

• 证明:

归纳基础:

- 当 $E = \varepsilon$ 时

$$h(\mathbf{L}(\varepsilon)) = h(\{\varepsilon\}) = \{\varepsilon\} = \mathbf{L}(\varepsilon) = \mathbf{L}(h(\varepsilon))$$

- 当 $E = \emptyset$ 时

$$h(\mathbf{L}(\emptyset)) = h(\emptyset) = \emptyset = \mathbf{L}(\emptyset) = \mathbf{L}(h(\emptyset))$$

- $\forall a \in \Sigma$, 当 $E = \mathbf{a}$ 时

$$h(\mathbf{L}(\mathbf{a})) = h(\{a\}) = \{h(a)\} = \mathbf{L}(h(a)) = \mathbf{L}(h(\mathbf{a}))$$

- 归纳递推：假设对正则表达式 F , G 分别有

$$\mathbf{L}(h(F)) = h(\mathbf{L}(F)), \quad \mathbf{L}(h(G)) = h(\mathbf{L}(G))$$

- 当 $E = F + G$ 时:

$$h(\mathbf{L}(F + G)) = h(\mathbf{L}(F) \cup \mathbf{L}(G))$$

$$= h(\mathbf{L}(F)) \cup h(\mathbf{L}(G))$$

$$= \mathbf{L}(h(F)) \cup \mathbf{L}(h(G))$$

$$= \mathbf{L}(h(F) + h(G))$$

$$= \mathbf{L}(h(F + G))$$

正则表达式的加

h 作用在每个集合的串上

归纳假设

正则表达式的加

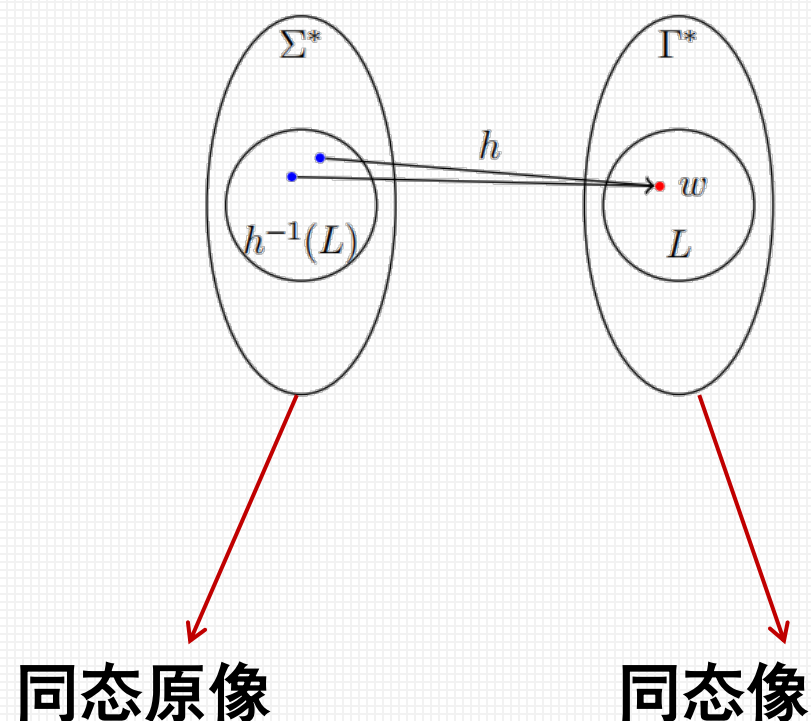
$h(F + G)$ 的定义

- 当 $E = FG$ 时: 略
- 当 $E = F^*$ 时: 略

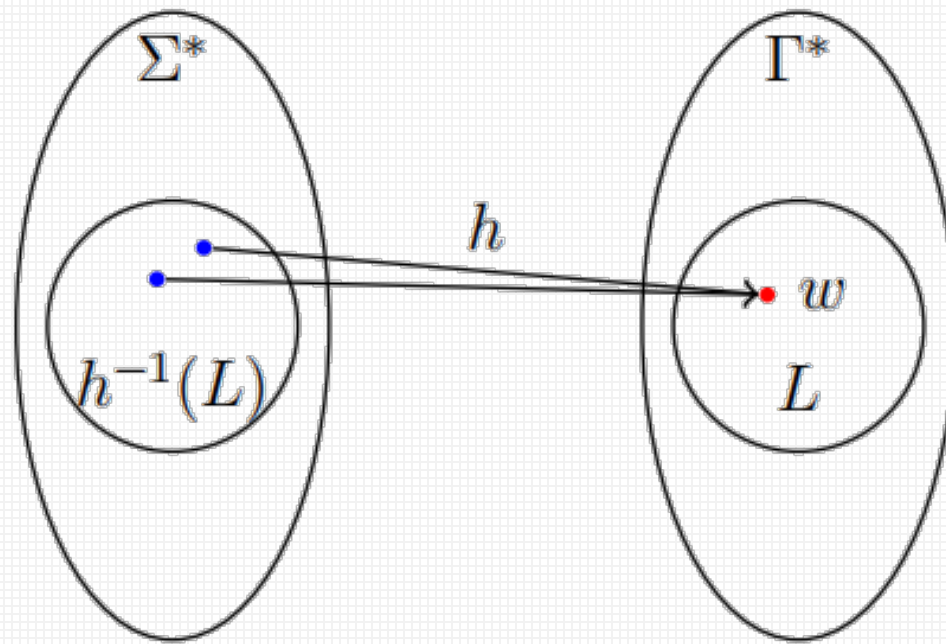
- 正则语言经某些**运算**后得到的新语言仍保持正则，称正则语言在这些运算下**封闭**。
- 正则语言 L 和 M ，在这些运算下封闭：
 - 并: $L \cup M$ ，连接: $L \cdot M$ ，闭包: L^*
 - 补: $\bar{L}$
 - 交: $L \cap M$
 - 差: $L - M$
 - 反转: $L^R = \{w^R \mid w \in L\}$
 - 同态: $h(L) = \{h(w) \mid w \in L\}$, 同态: $h: \Sigma \rightarrow \Gamma^*$
 - 逆同态: $h^{-1}(L) = \{w \in \Sigma^* \mid h(w) \in L \subset \Gamma^*, \text{同态: } h: \Sigma \rightarrow \Gamma^*\}$

- 定义：若 h 是字母表 Σ 到 Γ 的同态，且 L 是 Γ 上的语言，那么使 $h(w) \in L$ 的 w ($w \in \Sigma^*$) 的集合，称为语言 L 的 h 逆，记为 $h^{-1}(L)$.

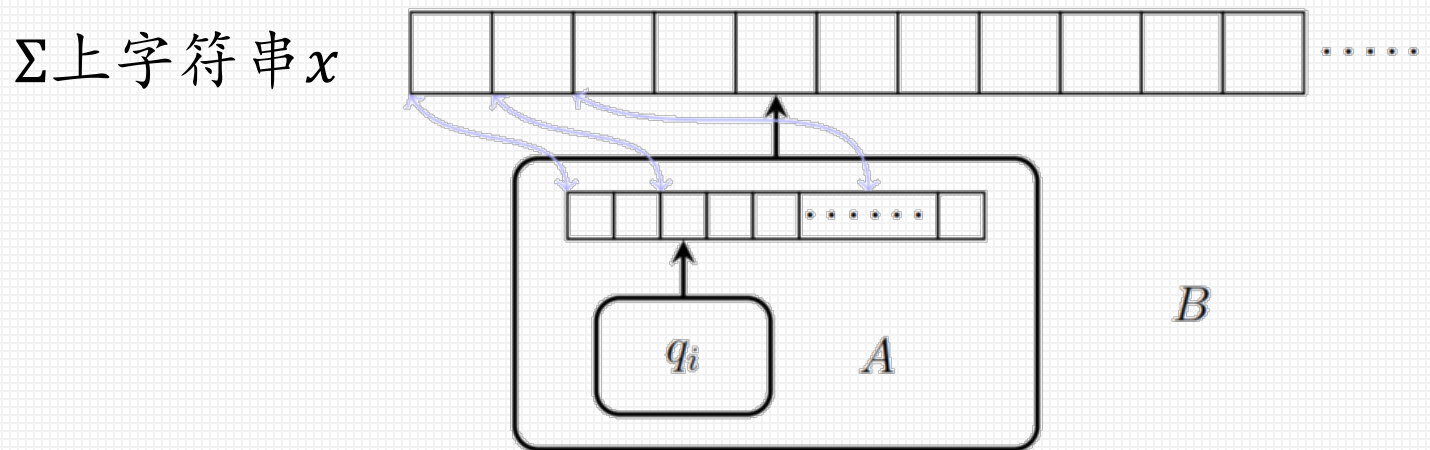
$$h^{-1}(L) = \{w \in \Sigma^* \mid h(w) \in L\}.$$



- 定理：如果 h 是字母表 Σ 到字母表 Γ 的同态， L 是 Γ 上的正则语言，那么 $h^{-1}(L)$ 也是正则语言。

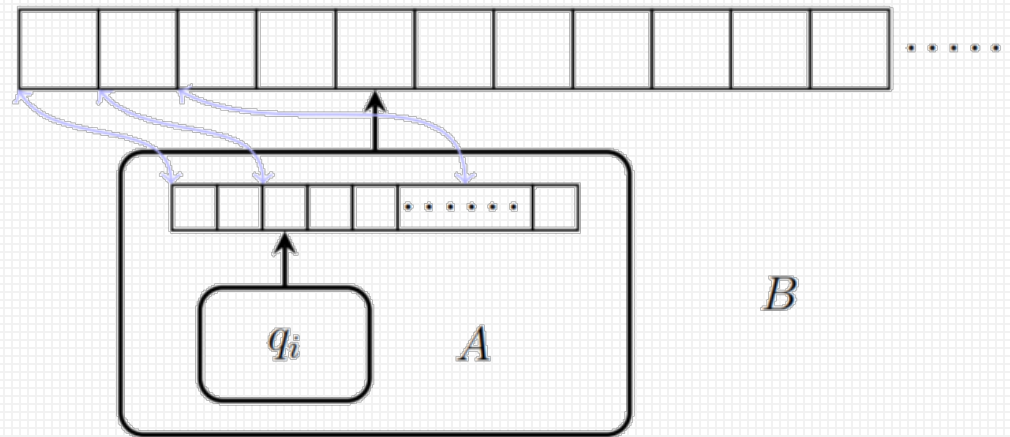


- 证明：由 L 的 DFA $A = (Q, \Gamma, \delta, q_0, F)$ ，构造识别 $h^{-1}(L)$ 的 DFA $B = (Q, \Sigma, \delta', q_0, F)$ ，



- 将A整个放入B的控制器中，将其输入带看作B控制器中的一个缓冲区；
- B每次读入一个字符 a ，在 h 的作用下映射为A输入带的一个字符串 $h(a)$ ；
- A扫描当前该字符串，进行状态转移；
- B扫描完整个字符串 x ，A所加载的字符串 w 就是 x 的像， $h(x) = w$ ；
- 若A接受 w ，则B接受 x 。让B的接受状态等于A接受状态即可。

- 证明：由 L 的 DFA $A = (Q, \Gamma, \delta, q_0, F)$ ，构造识别 $h^{-1}(L)$ 的 DFA $B = (Q, \Sigma, \delta', q_0, F)$ ，



- B 读入一个字符，相当于 A 读入一个字符串，因此 B 的 δ' 可由 A 的 $\hat{\delta}$ 定义：

$$\delta'(q, a) = \hat{\delta}(q, h(a))$$

- 为证明 $L(B) = h^{-1}(L)$, 先证明 $\hat{\delta}'(q, w) = \hat{\delta}(q, h(w))$, $w \in \Sigma^*$ 。

- 对 $|w|$ 归纳, 往证 $\hat{\delta}'(q, w) = \hat{\delta}(q, h(w))$
- 归纳基础: 若 $w = \varepsilon$, $\hat{\delta}(q, h(\varepsilon)) = \hat{\delta}(q, \varepsilon) = q = \hat{\delta}'(q, \varepsilon)$,
- 归纳递推: 若 $w = xa$,

$$\begin{aligned}
 \hat{\delta}'(q, xa) &= \delta'(\hat{\delta}'(q, x), a) \\
 &= \delta'(\hat{\delta}(q, h(x)), a) \\
 &= \hat{\delta}(\hat{\delta}(q, h(x)), h(a)) \\
 &= \hat{\delta}(q, h(x)h(a)) \\
 &= \hat{\delta}(q, h(xa))
 \end{aligned}$$

$\hat{\delta}'$ 的定义

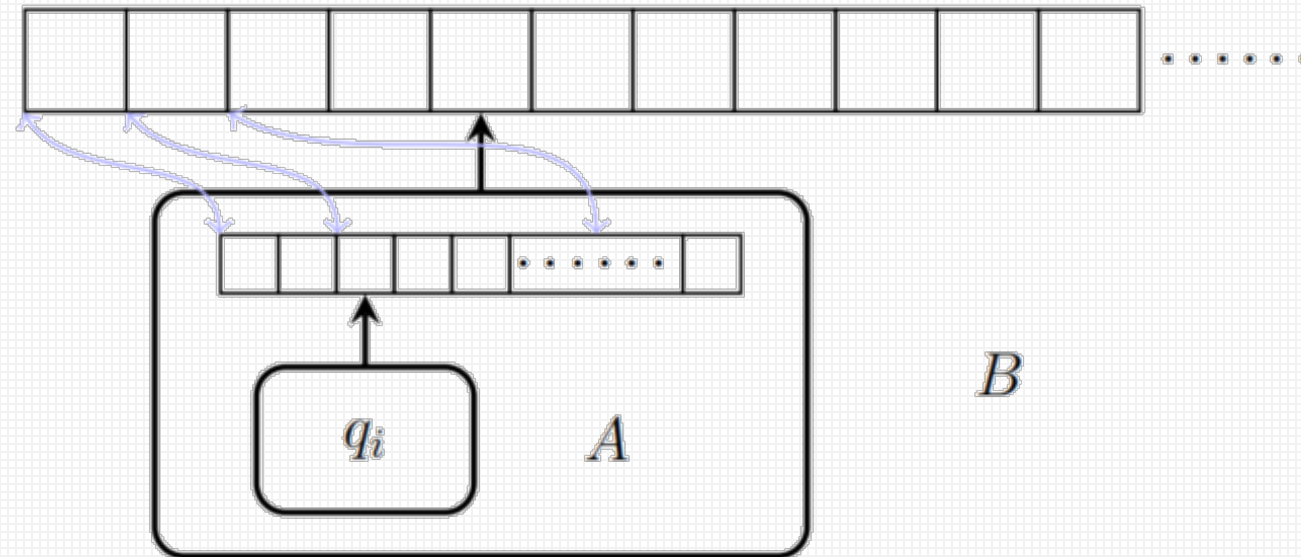
归纳假设

δ' 构造 $\delta'(q, a) = \hat{\delta}(q, h(a))$

$\hat{\delta}$ 性质

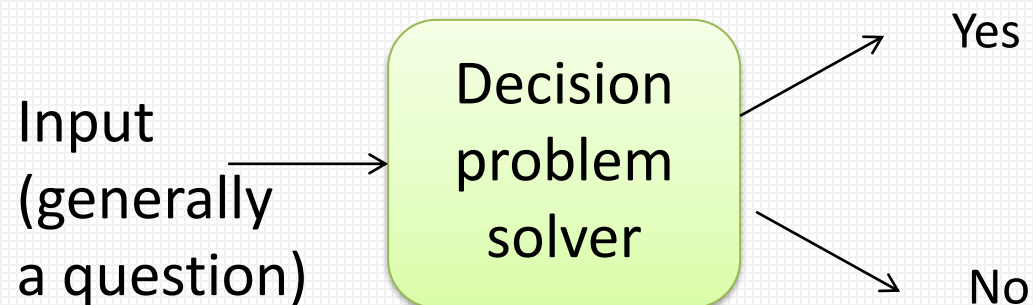
同态作用

- 已证 $\hat{\delta}'(q, xa) = \hat{\delta}(q, h(xa))$ 。
- 所以 $\forall w \in \Sigma^*, \hat{\delta}'(q_0, w) = \hat{\delta}(q_0, h(w)) \in F$
 - 即 w 被 B 接受当且仅当 $h(w)$ 被 A 接受, B 是识别 $h^{-1}(L)$ 的 DFA。
- 因此 $h^{-1}(L)$ 是正则的。



- 正则语言的泵引理
- 正则语言的封闭性
- **正则语言的判定性质**
- 自动机的最小化

- 判定问题 (decision problem) :



正则语言的判定性质 (Decision Properties)

- **正则语言中典型的3个判定问题：**
 - 以某种形式化模型描述的语言是否为空(empty)? 是否无穷(infinite)?
 - 以两种方式描述的语言, 是否是相同的? — 语言的等价性(equivalence)。
 - 某个特定的串 w 是否属于所描述的语言?
- **我们想知道的是, 回答这类问题的具体算法, 是否存在。**

- **定理：具有 n 个状态的有穷自动机 M 接受的集合 S ：**
 - S 是**非空**(nonempty)的，当且仅当 M 接受某个长度小于 n 的串；
 - S 是**无穷**(infinite)的，当且仅当 M 接受某个长度为 m 的串， $n \leq m < 2n$ 。
- **所以，对于正则语言：**
 - 存在算法，判断其是否为空，只需检查全部长度小于 n 的串；
 - 存在算法，判断其是否无穷，只需检查全部长度由 n 到 $2n - 1$ 的串。

- 1. 证明 非空性(non-emptiness):

必要性: 显然成立。

证明 S 是**非空**(nonempty)的, 当且仅当 M 接受某个长度小于 n 的串。

充分性:

- 如果 S 非空, 设 w 是 M 接受的串中长度最小的串
- 如果 $|w| \geq n$, 则由泵引理 $w = xyz$, 且 xz 也会被 M 被接受, 即有比 w 更短的串会被 M 接受, 但是 w 已经是最短的串, **矛盾**。
- 因此, $|w| < n$

S 是**无穷**(infinite)的, 当且仅当 M 接受某个长度为 m 的串, $n \leq m < 2n$ 。

• 2. 证明 无穷性

必要性: 由泵引理, 显然成立。

充分性:

- 如果 S 无穷, 假设没有长度 n 到 $2n - 1$ 之间的串;
- 取 $w \in L(M)$ 是长度 $\geq 2n$ 的最短的串;
- 由泵引理 $w = xyz$, 且 M 会接受更短的串 xz ;
- 因此, 或者 w 不是长度最小的, 或者长度 n 到 $2n - 1$ 之间有被接受的串,
- 因此假设不成立。

- **定理:**存在算法, 判定两个有穷自动机是否等价 (接受语言相同)。
- **证明:**
 - 设 M_1 和 M_2 是分别接受 L_1 和 L_2 的有穷自动机;
 - 则 $(L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2)$ 是正则的, 所以可被某个有穷自动机 M_3 接受;
 - 这里, M_3 接受某个串, 当且仅当 $L_1 \neq L_2$;
 - 由于存在算法判断 $L(M_3)$ 是否为空, 因此得证。

- 定理: 存在算法, 判定两个有穷自动机是否等价 (接受语言相同)。
- 证明:
 - 设 M_1 和 M_2 是分别接受 L_1 和 L_2 的有穷自动机;
 - 则 $(L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2)$ 是正则的, 所以可被某个有穷自动机 M_3 接受;
 - 这里, M_3 接受某个串, 当且仅当 $L_1 \neq L_2$;
 - 由于存在算法判断 $L(M_3)$ 是否为空, 因此得证。

也可以通过把 M_1 和 M_2 转化成最小化的DFA(minimal DFA), 然后看其是否等价。

- 正则语言的泵引理
- 正则语言的封闭性
- 正则语言的判定性质
- **自动机的最小化**

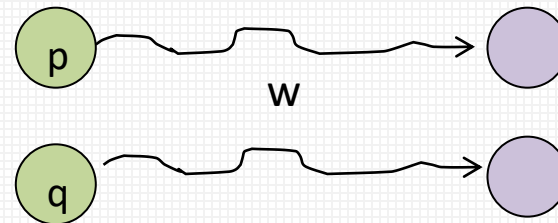
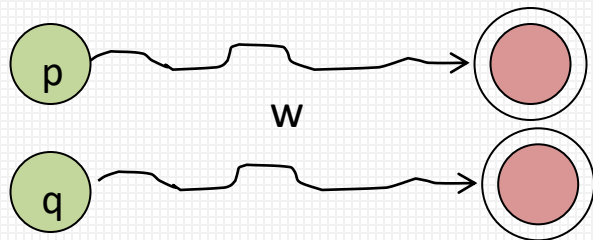
- DFA的最小化: 给定一个DFA, 可以找出一个等价的 (即描述同一语言) 具有最小状态数的DFA。
- 描述某个语言L的最小化的DFA (minimal DFA) 是唯一的 (unique)

- 定义: DFA $A = (Q, \Sigma, \delta, q_0, F)$ 中两个状态 p 和 q , 对 $\forall w \in \Sigma^*$:
$$\hat{\delta}(p, w) \in F \iff \hat{\delta}(q, w) \in F$$

则称这两个状态是等价的 (equivalent)。

也就是说, 如果两个状态是等价的, 则从 p 开始而被接受的串 w , 从 q 开始也会被接受; 从 p 开始而被拒绝的串 w , 从 q 开始也会被拒绝。

- 等价性只要求 $\hat{\delta}(p, w)$ 和 $\hat{\delta}(q, w)$ 同时在或不在 F 中, 而不必是相同状态。

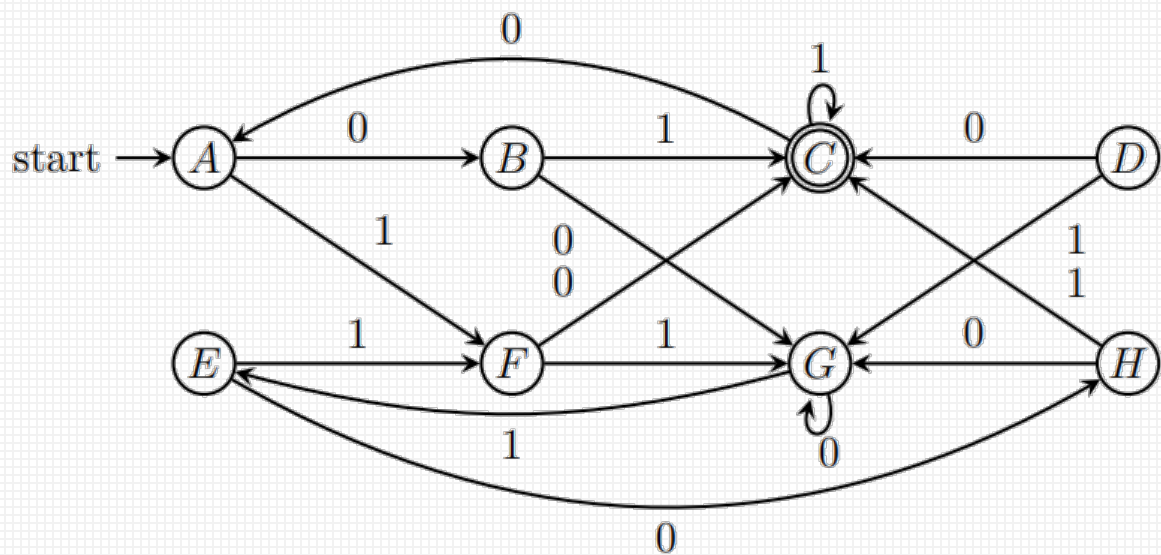


- 不等价的状态则称为可区分的 (distinguishable)
- 对于可区分的两个状态 p 和 q , 至少存在一个符号串 w , 使得 $\hat{\delta}(p, w)$ 和 $\hat{\delta}(q, w)$ 其中一个到达接受状态 (accepting), 另外一个不能到达接受状态 (not accepting)

- 填表算法 (table filling) 递归寻找DFA中全部的可区分状态对:
 - 基础: 如果 $p \in F$ 而 $q \notin F$, 则 $[p, q]$ 是可区分的;
 - 递推: 如果某个 $a \in \Sigma$, 有 $[r = \delta(p, a), s = \delta(q, a)]$ 是可区分的, 则 $[p, q]$ 是可区分的。

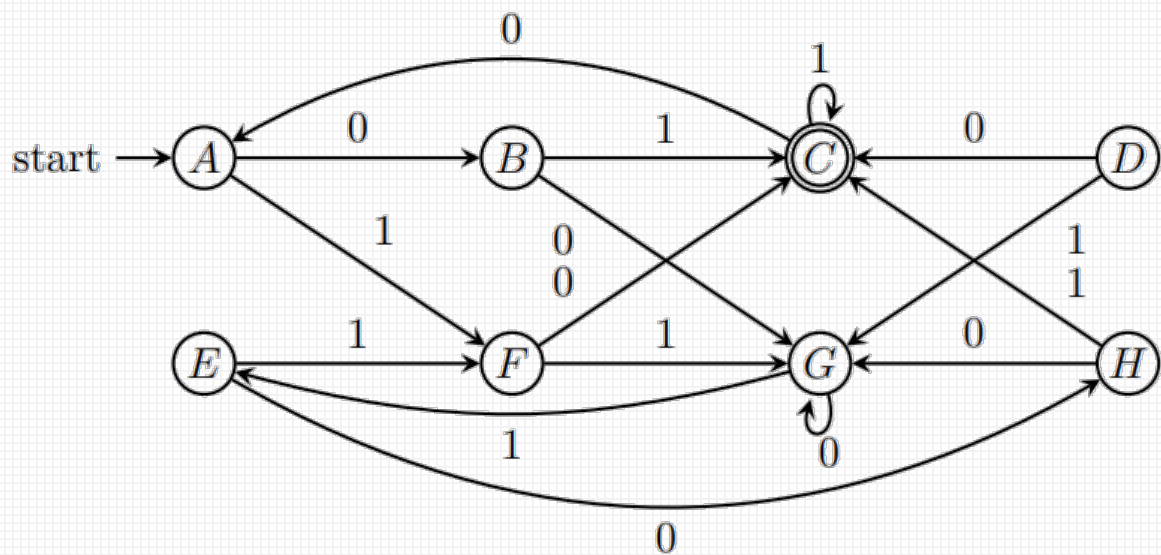
定理: 如果填表算法不能区分两个状态, 则这两个状态是等价的.

- 例：用填表算法找到如图DFA中全部可区分状态对。



B							
C							
D							
E							
F							
G							
H							
	A	B	C	D	E	F	G

- 例：用填表算法找到如图DFA中全部可区分状态对。

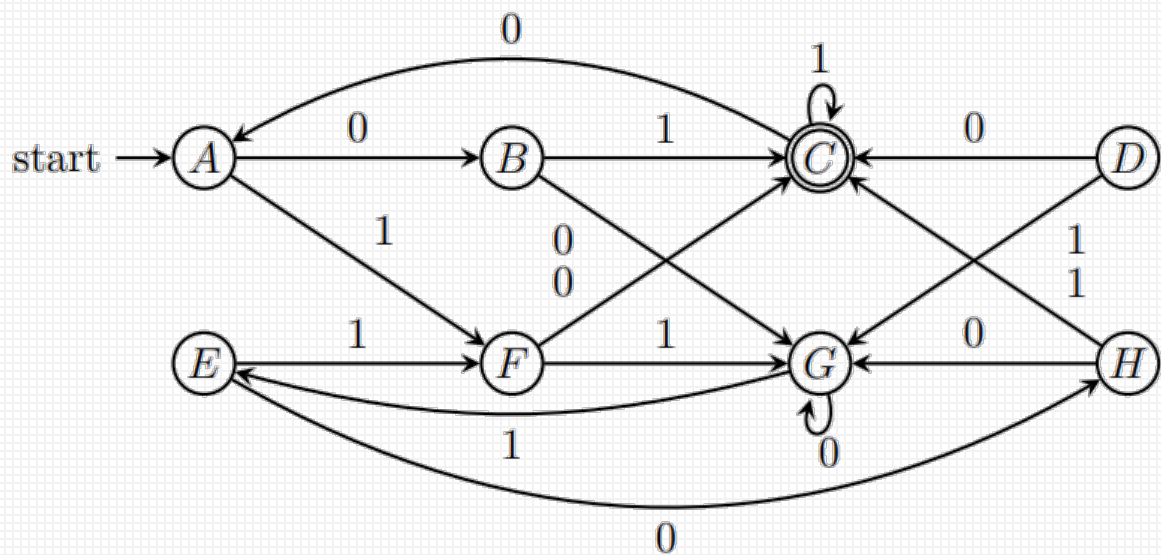


B							
C							
D							
E							
F							
G							
H							
	A	B	C	D	E	F	G

- (1)直接标记终态和非终态之间的状态对
- (2)标记所有经过字符0到达终态和非终态的状态对
- (3)标记所有经过字符1到达终态和非终态的状态对

× 表示不等价，即可区分。

- 例：用填表算法找到如图DFA中全部可区分状态对。



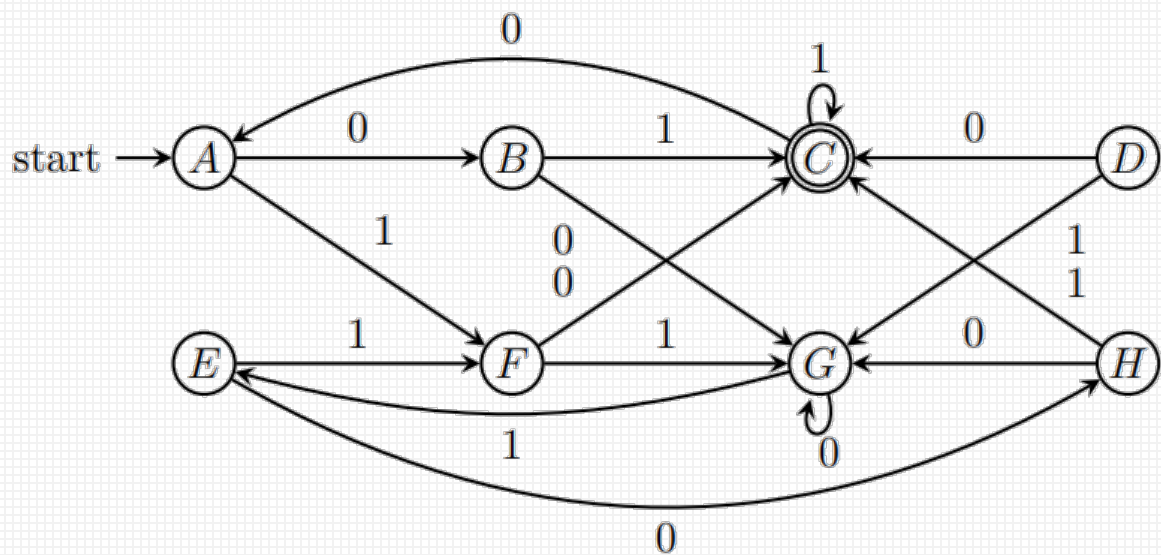
B							
C							
D							
E							
F							
G							
H							
	A	B	C	D	E	F	G

(1)直接标记终态和非终态之间的状态对:

$$\{C\} \times \{A, B, D, E, F, G, H\}$$

×表示不等价，即可区分。

- 例：用填表算法找到如图DFA中全部可区分状态对。



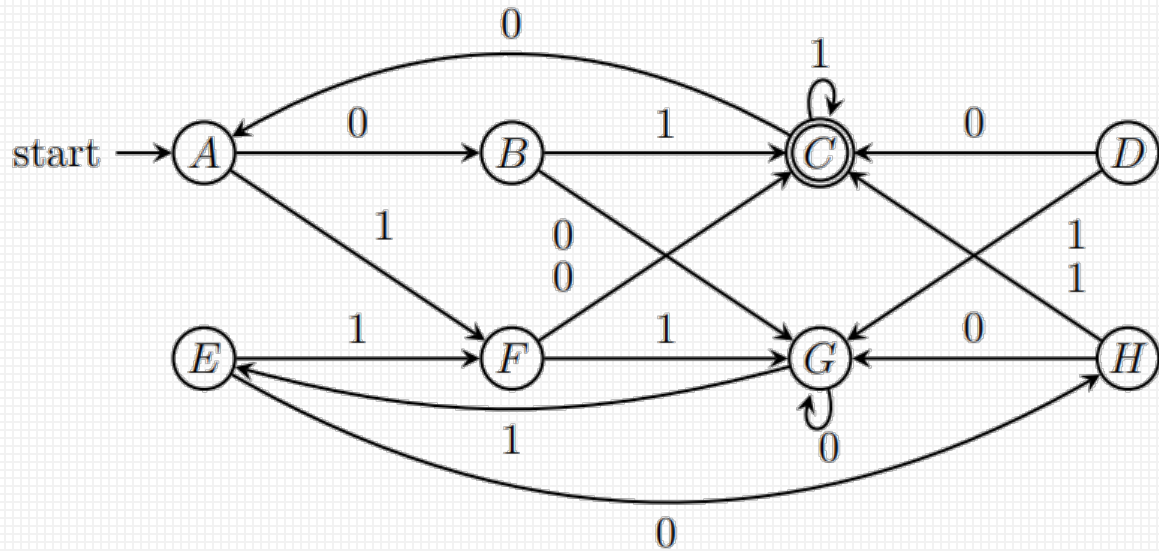
B							
C	×	×					
D			×				
E			×				
F			×				
G			×				
H			×				
	A	B	C	D	E	F	G

(1)直接标记终态和非终态之间的状态对:

$$\{C\} \times \{A, B, D, E, F, G, H\}$$

×表示不等价，即可区分。

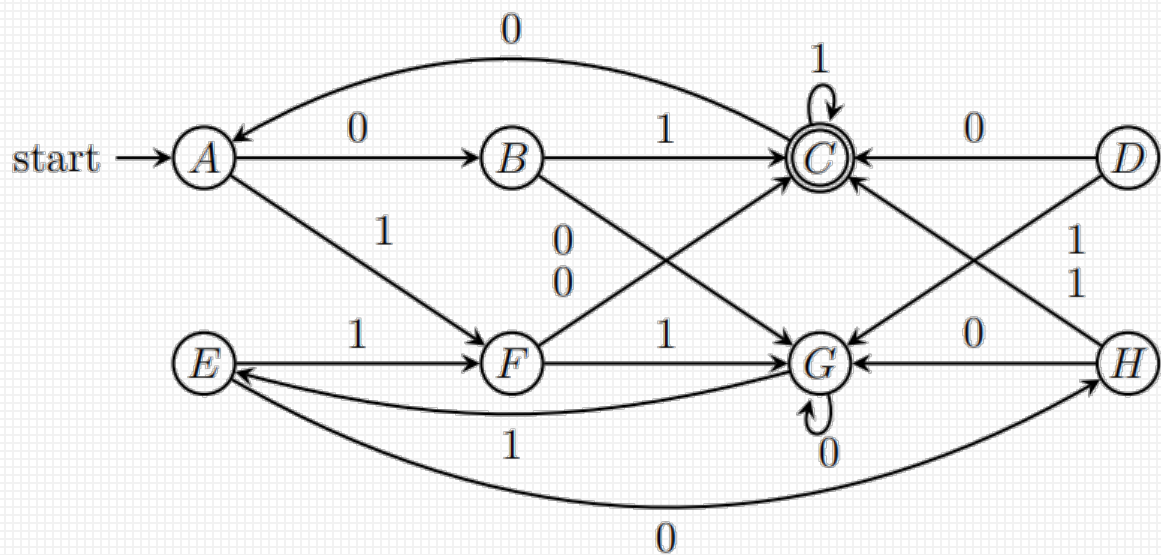
- 例：用填表算法找到如图DFA中全部可区分状态对。



B							
C	×	×					
D	×	×	×				
E			×	×			
F	×	×	×		×		
G			×	×		×	
H			×	×		×	
	A	B	C	D	E	F	G

(2) 标记所有经过字符0到达终态和非终态的状态对:
 $\{D, F\} \times \{A, B, E, G, H\}$

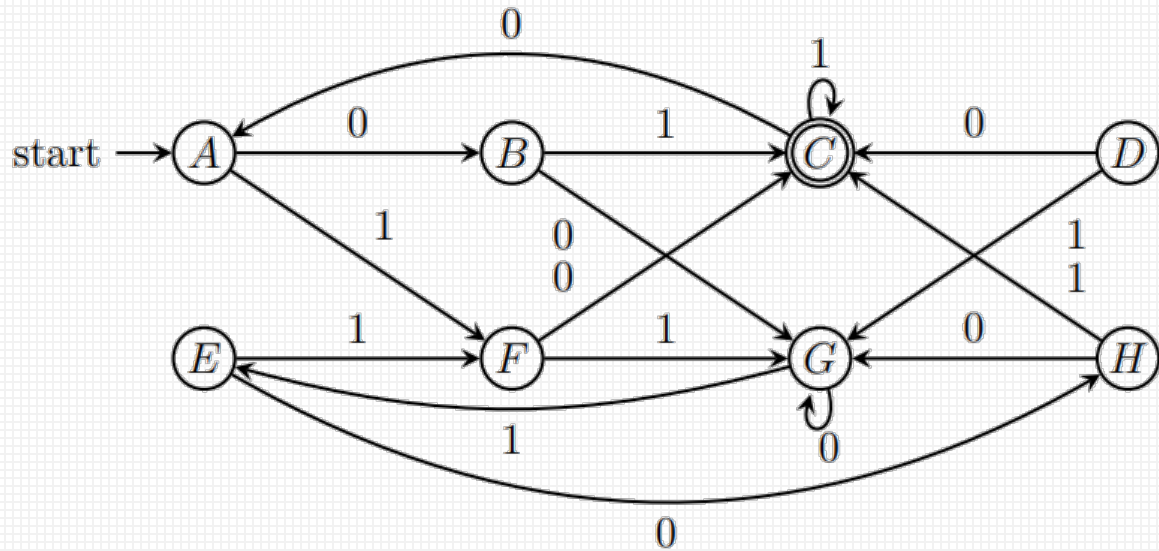
- 例：用填表算法找到如图DFA中全部可区分状态对。



B	×						
C	×	×					
D	×	×	×				
E		×	×	×			
F	×	×	×		×		
G		×	×	×		×	
H	×		×	×	×	×	×
	A	B	C	D	E	F	G

(3) 标记所有经过字符1到达终态和非终态的状态对：
 $\{B, H\} \times \{A, D, E, F, G\}$

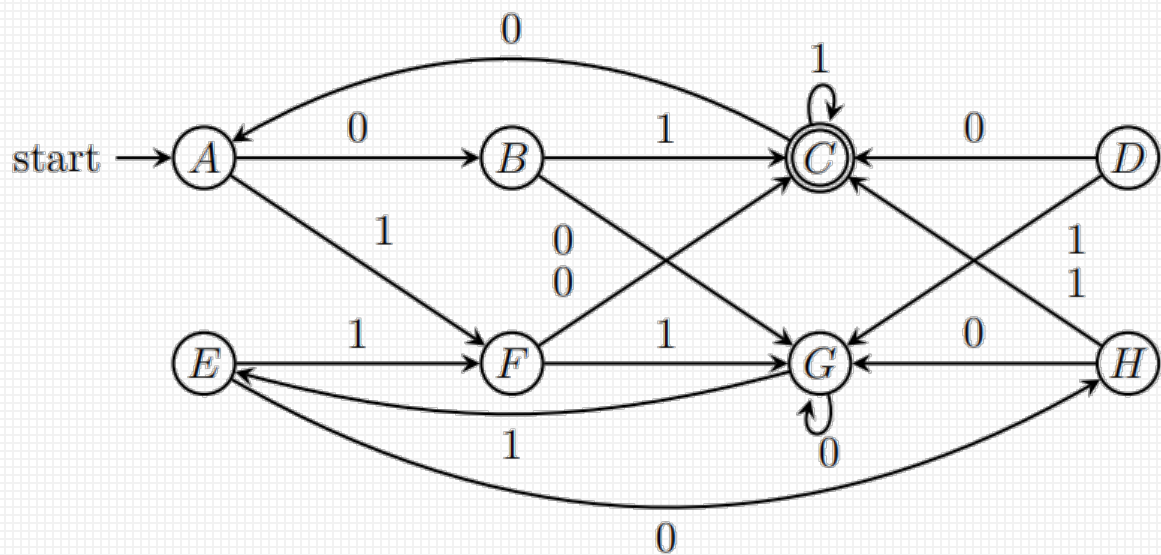
- 例：用填表算法找到如图DFA中全部可区分状态对。



(4)此时还有 $[A, E]$, $[A, G]$, $[B, H]$, $[D, F]$, $[E, G]$ 未标记, 只需逐个检查

B	×						
C	×	×					
D	×	×	×				
E		×	×	×			
F	×	×	×		×		
G		×	×	×		×	
H	×		×	×	×	×	×
	A	B	C	D	E	F	G

- 例：用填表算法找到如图DFA中全部可区分状态对。

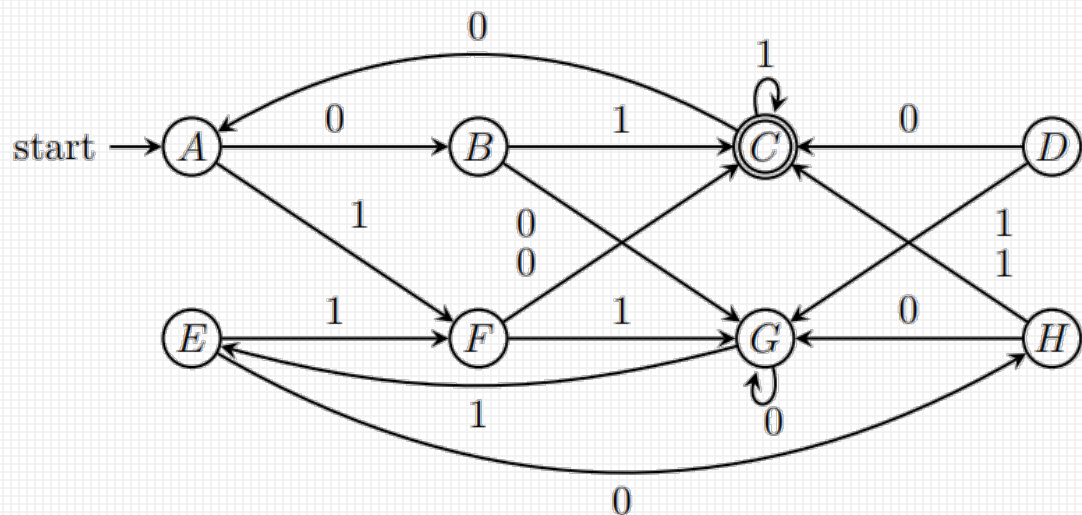


B	×						
C	×	×					
D	×	×	×				
E		×	×	×			
F	×	×	×		×		
G	×	×	×	×	×	×	
H	×		×	×	×	×	×
	A	B	C	D	E	F	G

(4)此时还有 $[A, E]$, $[A, G]$, $[B, H]$, $[D, F]$, $[E, G]$ 未标记, 只需逐个检查

- $[A, G]$ 是可区分的, 因为经串01到可区分的 $[C, E]$
- $[E, G]$ 是可区分的, 因为经串10到可区分的 $[C, H]$

- 例：用填表算法找到如图DFA中全部可区分状态对。



B	×						
C	×	×					
D	×	×	×				
E		×	×	×			
F	×	×	×		×		
G	×	×	×	×	×	×	
H	×		×	×	×	×	×
	A	B	C	D	E	F	G

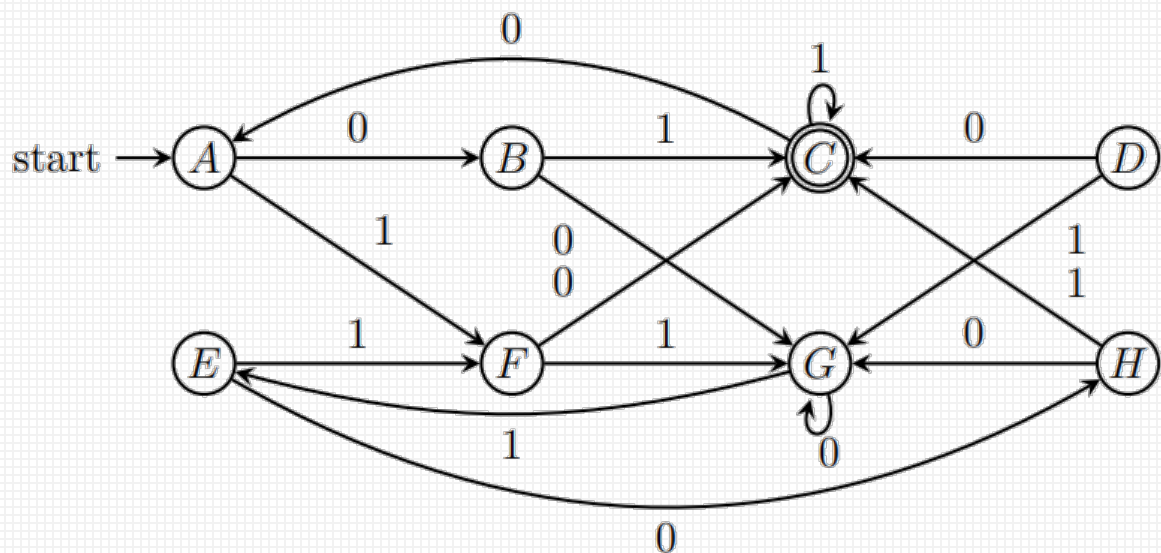
(5)此时还有 $[A, E]$, $[B, H]$, $[D, F]$ 未标记,

- 针对 $[A, E]$:

$(\delta(A, 1), \delta(E, 1)) = (F, F)$, 即消耗1时, 转移到同一个状态, 因此任何1开头的符号串都无法区分A和E, 因为最终都会到达同一个状态, 无法得出“可区分”结论。

$(\delta(A, 0), \delta(E, 0)) = (B, H)$, 消耗0时, 到达B和H, 而B和H在消耗0或者消耗1时, 都转移到同一个状态, 即G或者C, 因此亦无法区分A和E, 无法得出“可区分”结论

- 例：用填表算法找到如图DFA中全部可区分状态对。

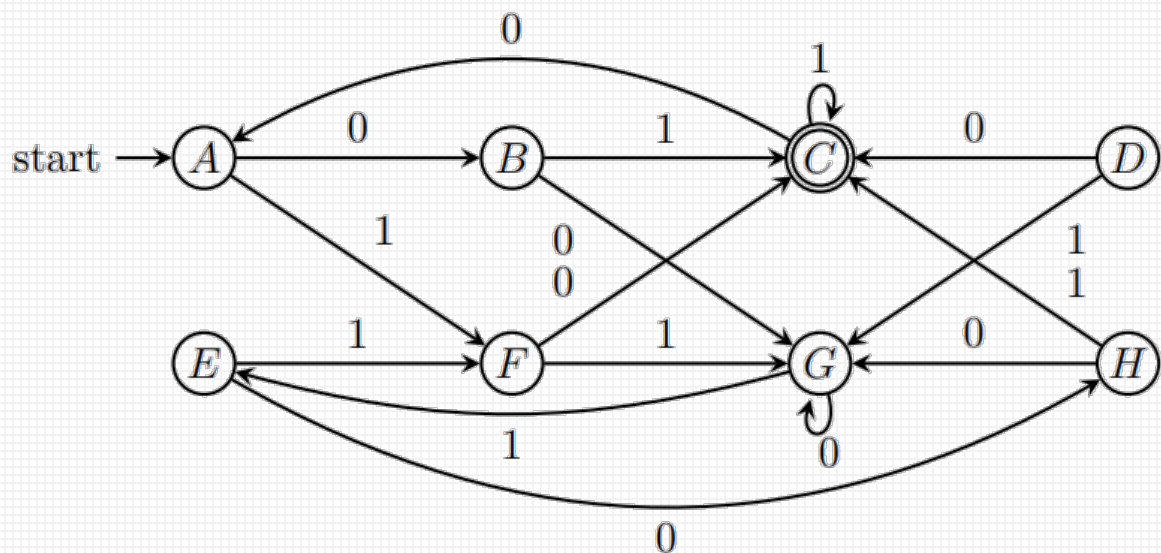


(5)此时还有 $[A, E]$, $[B, H]$, $[D, F]$ 未标记,

- 对 $[B, H]$ 和 $[D, F]$ 可作类似的分析, 得出它们都是不可区分的状态对。

B	×						
C	×	×					
D	×	×	×				
E		×	×	×			
F	×	×	×		×		
G	×	×	×	×	×	×	
H	×		×	×	×	×	×
	A	B	C	D	E	F	G

- 例：用填表算法找到如图DFA中全部可区分状态对。

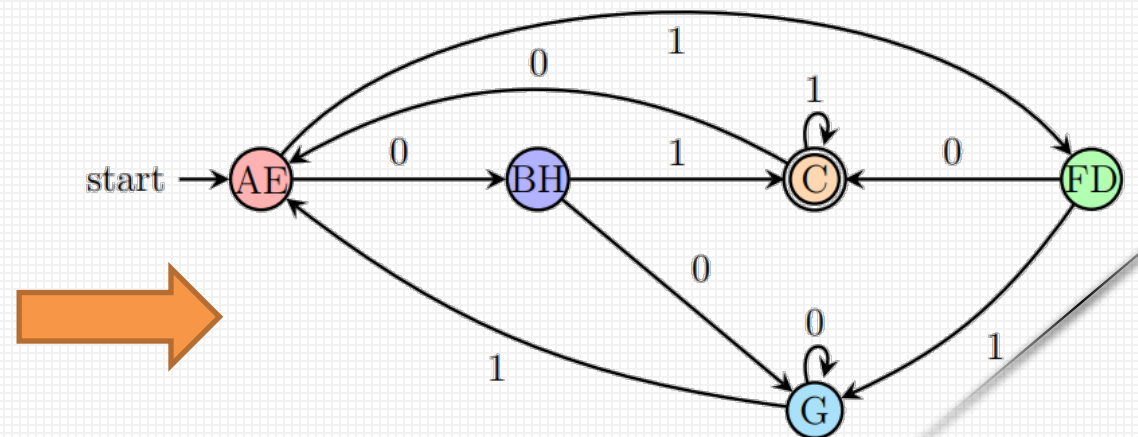
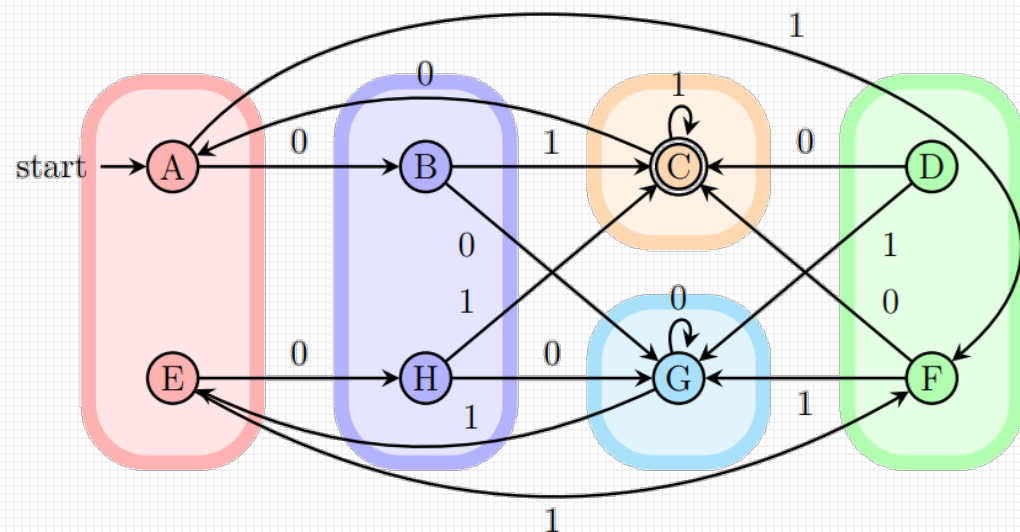


(5)因此， $[A, E]$, $[B, H]$, $[D, F]$ 都是等价的。

B	×						
C	×	×					
D	×	×	×				
E	=	×	×	×			
F	×	×	×	=	×		
G	×	×	×	×	×	×	
H	×	=	×	×	×	×	×
	A	B	C	D	E	F	G

- 根据填表算法取得的DFA $d1$ 状态的等价性，将状态集进行划分，得到不同的块；块中的状态是等价的。
- 利用块构造新的DFA $d2$ ， $d2$ 的开始状态为包含 $d1$ 开始状态的块， $d2$ 的接受状态为包含 $d1$ 接受状态的块，转移函数为块之间的转移
- 则 $d2$ 是 $d1$ 的最小化DFA。

等价状态对：[A, E], [B, H], [D, F]



- 正则语言的泵引理。
- 正则语言对有关运算的封闭性。
 - RL在并、乘、闭包、补、差、交、同态映射运算下是有效封闭的。
- 状态等价性和最小化DFA。